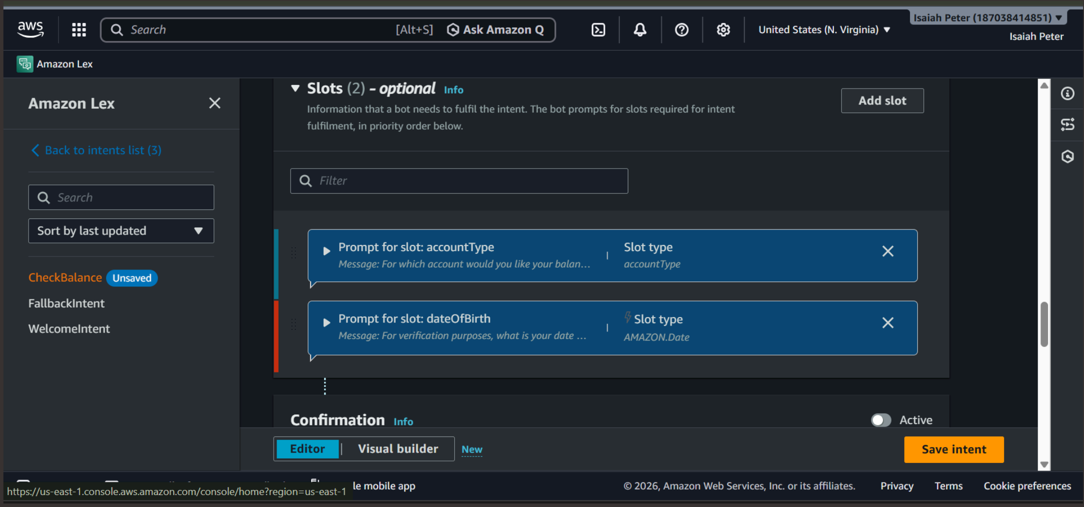
nextwork.ai

# Add Custom Slots to a Lex Chatbot

Isaiah Peter



# Introducing Today's Project!

In today's project, I used Amazon Lex to In today's project, I used Amazon Lex as the engine for my "BankerBot" to transform it into a functional and intelligent assistant. Specifically, I utilized the platform to: define intents, configure custom slots, Implement smart understanding, design robust error handling. Essentially, I used the Lex console to move from a basic chatbot shell to a sophisticated, intent driven interface that can reliably handle real world user interactions.

## What is Amazon Lex?

Amazon is the service I am using to build my BankerBot. It is a fully managed AI service from AWS that provides the same advanced technologies behind Alexa specifically Automatic Speech Recognition (ASR) to understand what is said and Natural Language Understanding (NLU) to interpret the intent behind the words. and it is useful because It allows me to move beyond simple command-line tools. I can build a bot that understands the context of a conversation, making the experience feel natural rather than robotic.

## One thing I didn't expect in this project was...

One thing I didn't expect was how much focus is placed on **error handling** and **slot validation** rather than just the "happy path" of the conversation. I initially thought building a chatbot would be primarily about teaching it to understand successful requests. However, I learned that a huge part of the work is actually designing how the bot gracefully recovers when a user makes a mistake like entering an invalid date of birth and guiding them back to the right path. It really highlighted that making an AI "smart" isn't just about what it can do correctly, but how effectively it manages the inevitable moments when a user gets it wrong.

## This project took me...

This project took me about 1 hour 20 minutes since I’ve been working on this iteratively as I learn, it’s been a manageable process. Because I am focusing on building the **CheckBalance** intent, implementing slot validation, and refining my error-handling strategies, I’ve spent about **a few hours of focused development time**. It’s been a great experience because that time wasn't spent wrestling with complex infrastructure or coding from scratch it was spent on the design logic, configuring the Lex console, and thinking through how to make the bot feel intuitive for my users. It’s been time well spent to ensure BankerBot is actually reliable.

# Slots

Slots are pieces of information that a chatbot needs to complete a user's request. Think of them as blanks that need to be filled in a form. For example, if the intent is to book a table at a restaurant, the chatbot needs specific details like: restaurant name, date, time, number of people. Amazon Lex provides many ready to use slot types for common information, like dates and times, but you can also create your own custom slot types to fit your specific needs.

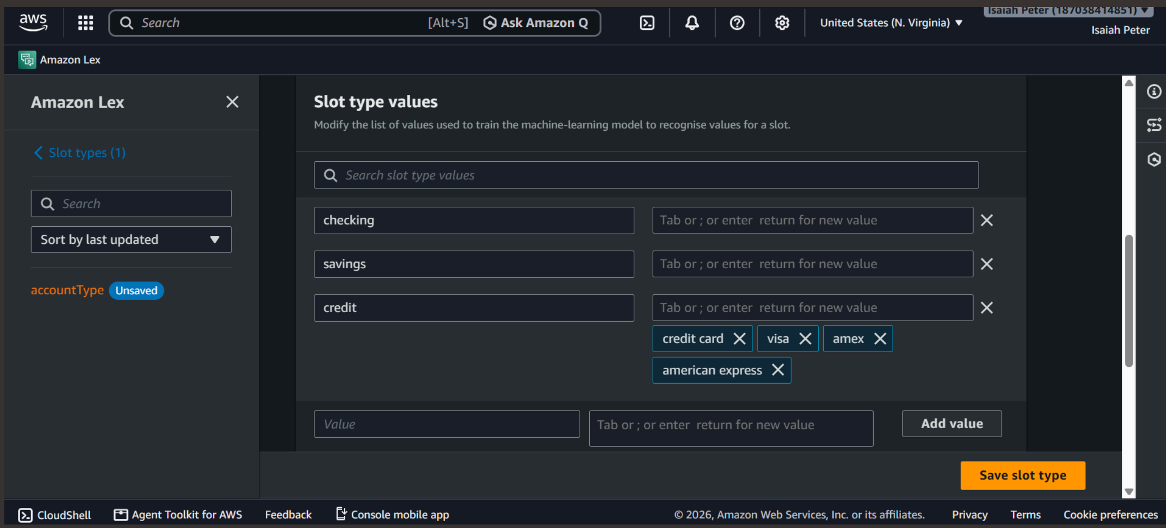
Having custom slots makes my bot more efficient and user friendly. By recognizing specific terms like "checking" or "savings" directly in a user's request, I reduce friction, minimize errors, and ensure the bot asks only the necessary follow-up questions, creating a smoother, more natural experience for my users.

In this project, I created a custom slot type for the BankerBot to check a user's bank balance. To make that happen, BankerBot will need to ask users for their bank account type and birthday for verification too. This will enable BankerBot to know what bank account types it should recognize.

# Isaiah Peter

NextWork Student

nextwork.ai



# Connecting slots with intents

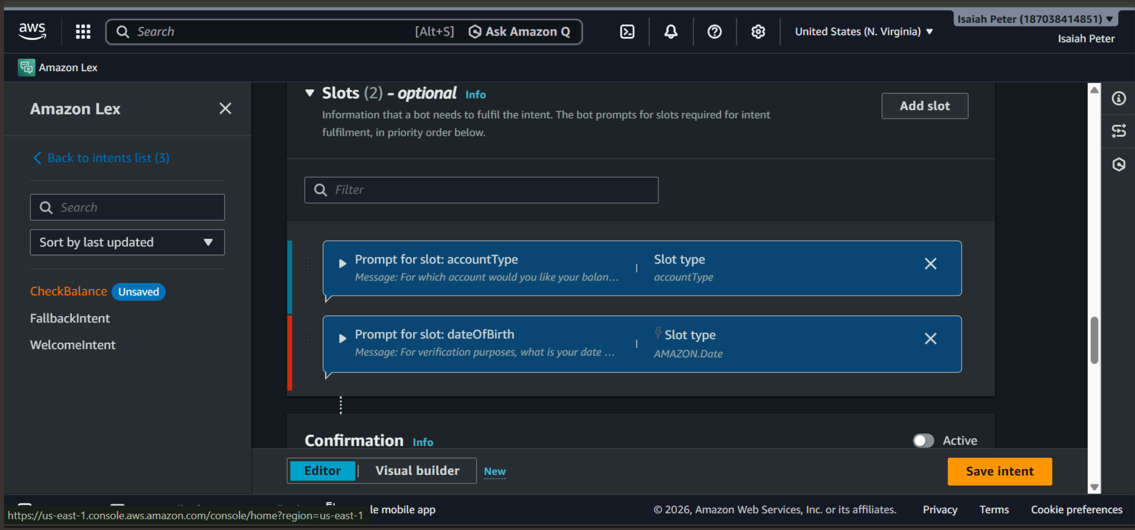
This slot type has restricted slot values, which means to makes sure that only the values that you specify will count as a valid accountType. Otherwise, Amazon Lex might use machine learning to accept other values that it frequently encounters from users. In this case, we don't want Lex to use machine learning to accept other values. We have a set list of bank account types that we offer - Checking, Savings, and Credit accounts and we don't offer any other bank account types. If Lex starts accepting other values outside of these three, users could end up having conversations about bank account types that the bot does not have access to or the bank does not actually offer. To prevent that from happening, I would restrict Lex to only acknowledge the bank account types has been restricted too.

The **CheckBalance** intent serves as the core instruction set for my chatbot, enabling it to recognize when a user wants to know their account balance. By setting this up, I am defining: * **Utterances:** The specific phrases I want my bot to listen for, such as "What is my balance?" or "Check my account." * **Slot Elicitation:** The logic that prompts the user to provide the necessary information (like the account type I just configured) so the bot can perform the request. * **Goal Completion:** The framework that allows the bot to take the collected information and eventually process or return the correct balance data to the user.

**Isaiah Peter**

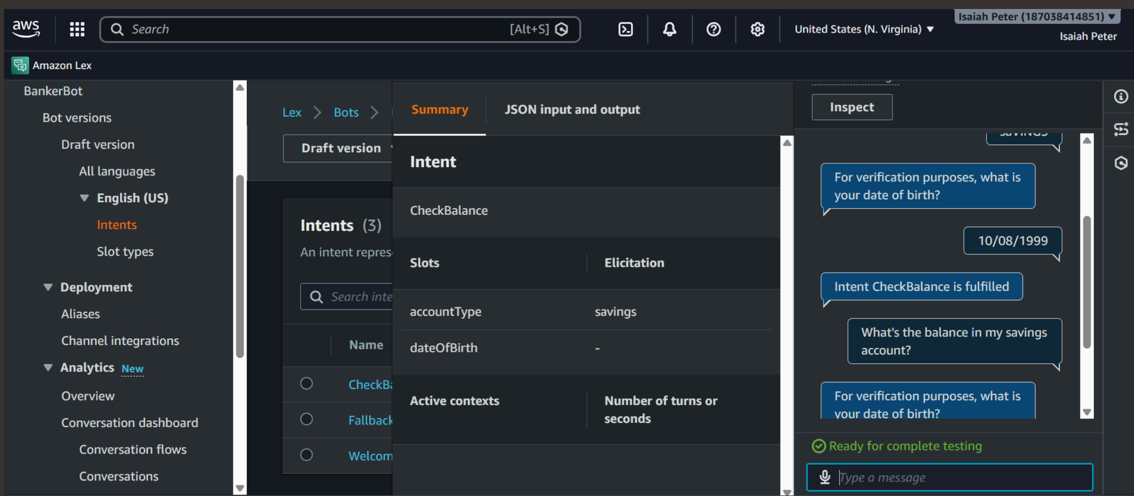
NextWork Student

nextwork.ai



# Slot values in utterances

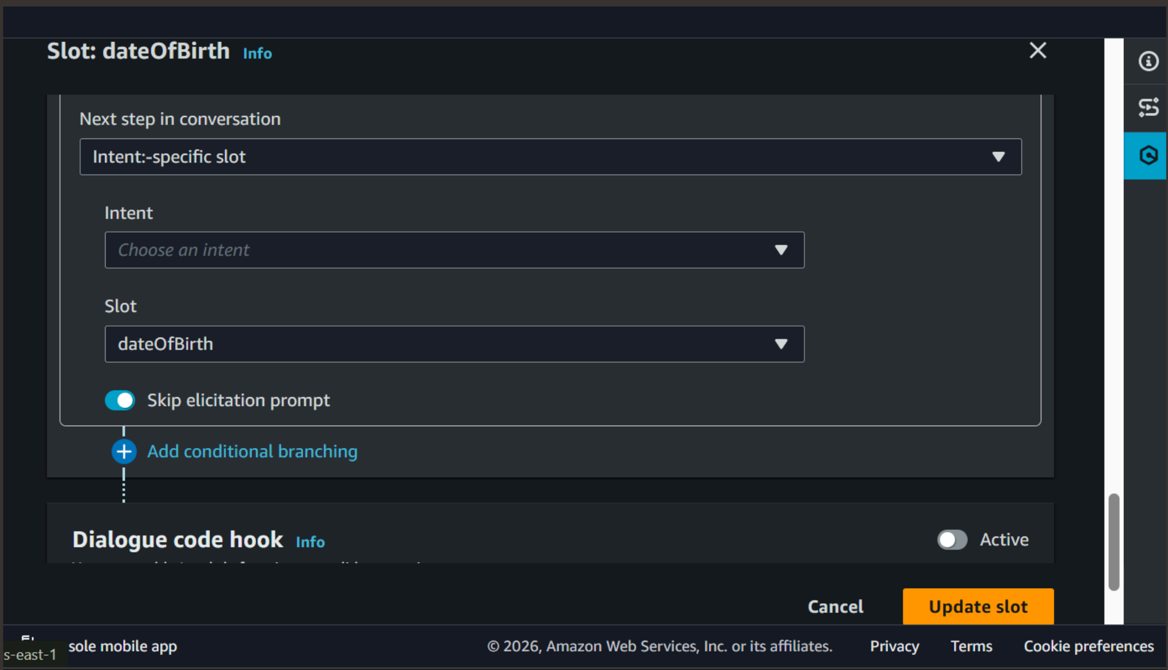
To include the slot values in my **CheckBalance** intent, I used the **slot syntax** in my sample utterances. By typing the slot name within curly braces, such as `Check my {accountType} balance`, I instructed Amazon Lex to treat that part of the phrase as a variable. This tells the bot that whenever a user says "Check my checking balance" or "Check my savings balance," it should automatically map the words "checking" or "savings" to the `accountType` slot I defined, allowing the bot to capture that specific information directly from the user's sentence.



# Handling failures in slot values

To set up slot variations, I configured **synonyms** for my slot values. Instead of just relying on the primary term, I defined alternative ways a user might refer to the same concept (for example, mapping "current account" or "checking account" to the "Checking" value). This allows my bot to understand different user expressions while still mapping them to the single, correct underlying data point.

To set up failure responses, I navigated to the **Slot Validation** or **Prompt** settings within my Lex intent. For each slot like the date of birth I configured a custom **retry message**. By defining these specific messages, I've programmed the bot to recognize when a user provides an invalid input or nothing at all, allowing it to automatically trigger a polite, conversational prompt to guide the user to try again. This ensures the conversation doesn't stall if the input doesn't match my validation rules.



# The place to learn & showcase your skills

Check out nextwork.ai for more projects